

2 Fundamento Teórico

2.1. Marco normativo aplicado a la especificación de requisitos

El marco normativo aplicado a la especificación de requisitos permite establecer criterios técnicos para documentar, revisar y validar un SRS. Las normas internacionales ayudan a evitar la improvisación, reducen la ambigüedad y facilitan que los requisitos sean comprendidos por todos los participantes del proyecto. En MundiPets, este marco es importante porque el sistema integra actores y procesos diversos, como propietarios, adoptantes, refugios, veterinarios, publicaciones, adopciones y citas médicas [1], [2]

La norma ISO/IEC/IEEE 29148:2018 es la referencia principal para la Ingeniería de Requerimientos. Esta norma define procesos y productos de información relacionados con requisitos de stakeholders, requisitos del sistema y requisitos de software. Su aplicación permite que los requisitos sean claros, necesarios, completos, verificables y trazables. En MundiPets, esta norma ayuda a redactar requisitos precisos, por ejemplo, para registrar mascotas, gestionar solicitudes de adopción o controlar citas veterinarias [1].

ISO/IEC/IEEE 15288:2023 aporta una visión del ciclo de vida del sistema. Esta norma permite comprender que los requisitos no pertenecen solo a la etapa inicial, sino que se relacionan con el desarrollo, operación, mantenimiento y evolución del sistema. En MundiPets, esto significa que cada requisito debe considerar no solo una pantalla o función, sino también los procesos, usuarios, datos, reglas y condiciones de operación que forman parte de la plataforma [2].

ISO/IEC/IEEE 12207:2017 se enfoca en los procesos del ciclo de vida del software. Aunque esta edición fue reemplazada posteriormente, se considera en este trabajo por estar solicitada en la guía de práctica. Su importancia está en conectar los requisitos con actividades como diseño, construcción, pruebas, mantenimiento y gestión de cambios. Para MundiPets, esto permite que los requisitos funcionales y no funcionales sirvan como base real para el desarrollo del software [3].

ISO/IEC 25010:2023 complementa la especificación de requisitos al proporcionar un modelo de calidad del producto de software. Esta norma permite definir atributos como seguridad, usabilidad, confiabilidad, eficiencia, mantenibilidad y compatibilidad. En MundiPets, estos atributos son necesarios para especificar requisitos no funcionales, por ejemplo, protección de datos de usuarios, facilidad de uso para adoptantes y disponibilidad del sistema para gestionar citas o publicaciones [4].

El SWEBOK v4.0 integra la Ingeniería de Requisitos dentro del cuerpo de conocimientos de la Ingeniería de Software y la relaciona con áreas como diseño, pruebas, calidad, mantenimiento y gestión de la configuración. Asimismo, el PMBOK orienta el control del alcance, la gestión de interesados, riesgos y cambios del proyecto, mientras que IREB CPRE proporciona conceptos, técnicas y criterios para la elicitación, documentación, validación y gestión de requisitos. Estas fuentes permiten que la especificación de MundiPets se desarrolle como un artefacto técnico integrado y no como una lista aislada de funcionalidades [5], [6], [7].

En conjunto, estas normas contribuyen a elaborar un SRS de calidad. ISO/IEC/IEEE 29148:2018 orienta la redacción de requisitos; ISO/IEC/IEEE 15288:2023 los ubica dentro del ciclo de vida del sistema; ISO/IEC/IEEE 12207:2017 los relaciona con procesos de software; e ISO/IEC 25010:2023 fortalece los requisitos de calidad. Para MundiPets, su integración permite producir una especificación completa, coherente, verificable y alineada con buenas prácticas internacionales.

2.2. Documentación de requisitos

Un requisito es una necesidad, capacidad, condición o restricción que el sistema debe cumplir para satisfacer a sus interesados. En Ingeniería de Requisitos, documentar adecuadamente estas necesidades permite transformar expectativas de usuarios en especificaciones técnicas que puedan diseñarse, implementarse y verificarse. En MundiPets, los requisitos provienen de necesidades asociadas al registro de mascotas, publicaciones, adopciones, cruce responsable, citas veterinarias, historial médico y control de permisos por rol [1], [8].

Los requisitos funcionales describen los servicios o acciones que el sistema debe realizar. En MundiPets, estos incluyen registrar mascotas, gestionar perfiles, publicar mascotas para adopción o cruce, enviar solicitudes, registrar historial médico, generar recordatorios, verificar identidad y clasificar imágenes. Estos requisitos responden a la pregunta sobre qué debe hacer el sistema y permiten delimitar el alcance funcional del proyecto [7], [8].

Los requisitos no funcionales describen atributos de calidad, restricciones técnicas o condiciones de operación. En MundiPets, se relacionan con seguridad, disponibilidad, rendimiento, usabilidad, exactitud del componente de inteligencia artificial y protección de datos sensibles. Por ejemplo, no basta con indicar que el sistema debe ser seguro; es necesario definir permisos, autenticación, control de acceso y criterios de verificación para proteger información personal y médica [4].

Un requisito de calidad debe ser claro, completo, consistente, verificable, factible, necesario y trazable. La claridad evita interpretaciones distintas; la completitud reduce omisiones; la consistencia previene contradicciones; la verificabilidad permite comprobar el cumplimiento; y la trazabilidad permite relacionar cada requisito con su fuente, modelo y prueba. En MundiPets, estos criterios son esenciales para evitar errores en procesos como adopción responsable, consulta de perfiles, revisión documental veterinaria y validación de identidad [1], [8].

La documentación de requisitos puede apoyarse en lenguaje natural estructurado, escenarios, casos de uso, reglas de negocio, modelos UML, DFD y matrices de trazabilidad. El lenguaje natural facilita la comunicación con los interesados, pero debe redactarse con precisión para evitar ambigüedades. En MundiPets, términos como “perfil completo”, “validar información médica” o “compatibilidad de cruza” deben definirse claramente para que puedan ser evaluados y modelados correctamente [1], [6].

Los escenarios Given–When–Then y la sintaxis EARS permiten estructurar requisitos mediante condiciones, eventos y respuestas esperadas. Por ejemplo, en MundiPets puede formularse: “When un usuario carga una imagen, the MundiPets system shall clasificarla como Aceptada, Rechazada o Pendiente de revisión”. Este tipo de redacción mejora la comprensión, la trazabilidad y la definición de criterios de prueba [1], [9].

2.3. Documento ERS/SRS según IEEE/ISO/IEC 29148:2018

El documento ERS/SRS es el artefacto formal donde se organizan los requisitos de software de manera clara, estructurada y verificable. Su propósito es describir qué debe hacer el sistema, qué restricciones debe cumplir, qué datos debe gestionar y qué atributos de calidad debe satisfacer. En MundiPets, el SRS consolida los requisitos relacionados con usuarios, roles, mascotas, publicaciones, solicitudes, citas veterinarias, documentos veterinarios e historial médico [1].

El SRS permite delimitar el alcance del sistema, facilitar acuerdos entre interesados, orientar el diseño, apoyar la planificación de pruebas y controlar los cambios. Además, reduce ambigüedades y ofrece una base común para analistas, modeladores, desarrolladores y evaluadores. En MundiPets, este documento evita que los actores interpreten de forma distinta procesos como adopción, cruza responsable o revisión documental veterinaria [1], [8].

Según ISO/IEC/IEEE 29148:2018, un SRS debe contener información suficiente sobre propósito, alcance, contexto, interesados, requisitos funcionales, requisitos no funcionales, restricciones, interfaces, supuestos, dependencias y criterios de verificación. Esta estructura puede adaptarse al proyecto, pero debe conservar coherencia,

trazabilidad y verificabilidad. En MundiPets, el SRS también se complementa con modelos de datos, funcionales y de comportamiento para representar mejor el sistema [1].

Un SRS de calidad debe ser correcto, completo, consistente, verificable, modificable y trazable. Estas características permiten que el documento sea útil durante todo el ciclo de vida del software. En MundiPets, la calidad del SRS permite comprobar si cada requisito está respaldado por una necesidad real, si posee modelos asociados y si puede validarse mediante pruebas o revisión con interesados [1], [8].

El SRS también permite detectar conflictos antes de la implementación. Por ejemplo, si un requisito permite publicar mascotas sin validación, pero otro exige control de identidad o privacidad, el conflicto debe resolverse en la especificación. De esta forma, el SRS actúa como instrumento técnico de control del alcance, consistencia y calidad del proyecto [1], [8].

Modelos de datos

Los modelos de datos representan la estructura de la información que el sistema debe almacenar, consultar, modificar y relacionar. En Ingeniería de Requisitos, estos modelos son importantes porque permiten identificar entidades, atributos, relaciones, claves y restricciones derivadas de los requisitos funcionales. En MundiPets, el modelo de datos organiza información sobre usuarios, roles, mascotas, historial médico, vacunas, documentos veterinarios, publicaciones, solicitudes y citas [7], [8].

El Modelo Entidad–Relación permite identificar los conceptos principales del dominio y sus vínculos. Una entidad representa un objeto relevante del sistema, mientras que sus atributos describen características necesarias para cumplir los requisitos. En MundiPets, entidades como Usuario, Mascota, HistorialMedico, Publicacion, SolicitudAdopcion, SolicitudCruza y CitaVeterinaria permiten estructurar la información esencial del sistema [8].

En el modelo de MundiPets, los actores Refugio, Veterinario, Propietario, Adoptante y Administrador no se modelan como entidades independientes, sino como roles especializados de la entidad Usuario. Esta decisión evita duplicidad de datos personales y permite que un mismo usuario pueda tener uno o varios roles mediante la entidad asociativa UsuarioRol. Con ello se mantiene una estructura más flexible y normalizada [8].

Las claves primarias identifican de manera única cada registro, mientras que las claves foráneas permiten relacionar entidades. En MundiPets, `id_mascota` identifica cada

mascota, id_usuario identifica cada usuario e id_publicacion permite vincular solicitudes de adopción con publicaciones. Esta estructura evita inconsistencias como citas sin mascota, solicitudes sin publicación o documentos veterinarios sin mascota asociada [7], [8].

La normalización permite reducir redundancia y prevenir anomalías de inserción, actualización o eliminación. La Tercera Forma Normal exige que los atributos no clave dependan únicamente de la clave primaria. En MundiPets, los datos de vacunas se almacenan en Vacuna y su aplicación se registra en MascotaVacuna; los datos del veterinario se mantienen en Usuario y no se repiten en CitaVeterinaria o DocumentoVeterinario. Esto mejora la integridad, consistencia y mantenibilidad del modelo [7], [8].

La transformación del modelo ER al diagrama de clases UML permite conectar el análisis de datos con el diseño orientado a objetos. Las entidades pueden convertirse en clases, los atributos en propiedades y las relaciones en asociaciones con multiplicidades. En MundiPets, clases como Usuario, Mascota, SolicitudAdopcion, CitaVeterinaria e HistorialMedico permiten representar la estructura principal del sistema. Esta transformación facilita avanzar desde la especificación hacia el diseño técnico [8].

2.4. Modelos funcionales

Los modelos funcionales representan los procesos que realiza el sistema, las entradas que recibe, las salidas que produce y los datos que transforma. En Ingeniería de Requerimientos, estos modelos permiten comprender qué hace el sistema sin definir todavía detalles de programación o tecnología. Para MundiPets, son útiles porque permiten representar procesos como registro de usuarios, gestión de mascotas, adopciones, publicaciones, citas veterinarias y historial médico [7], [8].

El Diagrama de Contexto, también conocido como DFD Nivel 0, representa el sistema como un único proceso central y muestra sus relaciones con entidades externas. En MundiPets, el proceso central puede denominarse “Gestionar plataforma MundiPets”. Las entidades externas serían propietario, adoptante, refugio, veterinario y administrador. Los flujos de datos incluirían datos de usuario, información de mascotas, solicitudes, citas, publicaciones y reportes [7], [8].

El DFD Nivel 1 descompone el proceso general en procesos más específicos. Para MundiPets, estos procesos pueden ser gestionar usuarios, gestionar mascotas, gestionar publicaciones, gestionar adopciones, gestionar citas veterinarias y gestionar historial médico. Esta descomposición permite verificar si cada requisito funcional tiene

cobertura dentro del modelo. También ayuda a detectar procesos innecesarios o requisitos que no han sido suficientemente definidos [8], [10]

El DFD esencial representa la lógica del sistema sin comprometerse con decisiones físicas de implementación. Su utilidad consiste en mostrar qué información entra, cómo se transforma y qué información sale. En MundiPets, un DFD esencial de adopción puede mostrar que el adoptante envía una solicitud, el sistema valida datos, registra la solicitud y notifica al refugio responsable. Esta vista facilita analizar el proceso antes de diseñar pantallas o base de datos [7], [8].

Los elementos principales de un DFD son procesos, flujos de datos, almacenes de datos y entidades externas. Los procesos transforman información; los flujos muestran movimiento de datos; los almacenes conservan información persistente; y las entidades externas interactúan con el sistema desde fuera de su frontera. En MundiPets, los almacenes principales pueden ser Usuarios, Mascotas, Solicitudes, Citas y historial médico [7], [8].

Los diagramas de actividades UML complementan los DFD porque muestran el orden de ejecución de actividades, decisiones y responsabilidades. Las swimlanes permiten separar acciones por actor, como adoptante, sistema y refugio. Las decisiones representan condiciones, mientras que fork y join permiten modelar actividades paralelas. En MundiPets, estos diagramas son útiles para representar solicitud de adopción, agendamiento de citas, publicación de mascotas y validación de información [10], [11].

2.5. Modelos de comportamiento

Los modelos de comportamiento describen cómo cambia el sistema ante eventos, condiciones o mensajes. Estos modelos son necesarios cuando existen estados relevantes, aprobaciones, cancelaciones, notificaciones o secuencias de interacción entre actores. En MundiPets, permiten representar el ciclo de vida de solicitudes de adopción, publicaciones, citas veterinarias y disponibilidad de mascotas. Por ello, complementan la especificación funcional y de datos [7], [8]

El diagrama de máquina de estados UML permite representar estados, eventos, acciones y transiciones. Un estado describe una condición significativa de un objeto; un evento provoca un cambio; una acción se ejecuta durante la transición; y una transición conecta un estado con otro. En MundiPets, una SolicitudAdopción puede pasar por estados como Pendiente, En revisión, Aprobada, Rechazada o Cancelada [8], [12].

Los estados compuestos permiten organizar procesos internos dentro de un estado general. Por ejemplo, el estado “En revisión” de una solicitud puede incluir validación de datos, revisión del refugio y decisión final. Esta representación ayuda a precisar reglas de negocio y evita cambios de estado incorrectos. En MundiPets, permite controlar que una solicitud no sea aprobada sin cumplir las condiciones definidas por el refugio responsable [8], [12].

El diagrama de secuencia UML representa la interacción temporal entre actores, objetos y componentes del sistema. Sus elementos principales son objetos participantes, líneas de vida y mensajes. En MundiPets, un diagrama de secuencia para agendar cita puede incluir al propietario, interfaz del sistema, servicio de citas, base de datos y veterinario. Este modelo permite observar el orden de comunicación y las responsabilidades de cada componente [8].

Los mensajes síncronos ocurren cuando el emisor espera una respuesta antes de continuar; por ejemplo, cuando el sistema consulta disponibilidad de un veterinario antes de registrar una cita. Los mensajes asíncronos permiten continuar el flujo sin esperar respuesta inmediata, como el envío de una notificación. Los fragmentos combinados alt y loop permiten representar alternativas y repeticiones. En MundiPets, alt puede usarse cuando existe o no disponibilidad, y loop cuando el usuario busca otro horario [12], [13].

Estos modelos fortalecen el SRS porque permiten especificar reglas dinámicas que no siempre son evidentes en requisitos textuales. En MundiPets, no basta con indicar que el sistema gestiona adopciones; también debe precisarse qué estados tiene una solicitud, quién puede modificarlos y qué acciones ocurren en cada transición. Esto mejora la verificabilidad, la trazabilidad y la consistencia de la especificación [1], [8], [12]

2.6. Interrelación entre las tres perspectivas de especificación

Las perspectivas de datos, funcional y comportamiento se complementan para construir una especificación completa del sistema. La perspectiva de datos define qué información existe y cómo se relaciona; la funcional explica qué procesos transforman esa información; y la de comportamiento muestra cómo cambian los estados ante eventos. Si se utiliza una sola perspectiva, el SRS puede quedar incompleto o presentar inconsistencias [1], [7], [8].

En MundiPets, esta complementariedad se evidencia en el proceso de adopción. La perspectiva de datos identifica entidades como Usuario, Mascota, Publicacion y SolicitudAdopcion. En este modelo, Refugio se gestiona como un rol especializado de Usuario. La perspectiva funcional muestra procesos como registrar solicitud, validar

información y notificar al refugio. La perspectiva de comportamiento define estados como Pendiente, En revisión, Aprobada, Rechazada o Cancelada [8], [13].

La integración de perspectivas mejora la calidad del SRS porque permite detectar omisiones y contradicciones. Si existe un requisito sobre historial de salud, el modelo de datos debe incluir HistorialMedico, el modelo funcional debe mostrar su gestión y el modelo de comportamiento debe representar eventos relacionados con su actualización. Así, cada requisito queda mejor sustentado y puede verificarse con mayor precisión [1], [7], [8].

La trazabilidad también se fortalece porque cada requisito puede vincularse con entidades, procesos, actividades, estados y pruebas. En MundiPets, un requisito sobre registrar observaciones veterinarias debe relacionarse con la entidad HistorialMedico, el proceso de gestión de salud, el rol Veterinario y los criterios de verificación. Esto evita requisitos sin cobertura o modelos sin justificación dentro del SRS [1], [8], [11].

La comunicación entre interesados mejora cuando se utilizan varias perspectivas de análisis. Los usuarios comprenden mejor los procesos y actividades; los desarrolladores requieren clases, datos y relaciones; y los evaluadores necesitan estados, escenarios y criterios de prueba. En MundiPets, esta integración facilita que propietarios, adoptantes, refugios, veterinarios y administradores validen el sistema desde sus propias necesidades [7], [8].

Bibliografia

- [1] ISO/IEC/IEEE, “29148:2018 — Requirements engineering,” 2018, *IEEE, Piscataway, NJ, USA*. doi: 10.1109/IEEESTD.2018.8559686.
- [2] ISO/IEC/IEEE, “15288:2023 — System life cycle processes,” 2023.
- [3] ISO/IEC/IEEE, “ISO/IEC/ IEEE 12207:2017 — Systems and software engineering-Software life cycle processes,” 2017.
- [4] ISO/IEC, “25010:2023 Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model,” 2023.
- [5] P. M. Institute, *A Guide to the Project Management Body of Knowledge (PMBOK® Guide) – Seventh Edition and The Standard for Project Management*. Newtown Square, PA: Project Management Institute, 2021.
- [6] IREB, “CPRE Foundation Level Syllabus v3.1,” 2023. [Online]. Available: <https://www.ireb.org>
- [7] H. Washizaki, *SWEBOK Guide v4.0*. IEEE Computer Society, 2024. [Online]. Available: <https://www.computer.org/education/bodies-of-knowledge/software-engineering>
- [8] Klaus. Pohl, *Requirements engineering : fundamentals, principles, and techniques*. Springer, 2025.
- [9] L. P. Binamungu and S. Maro, “Behaviour driven development: A systematic mapping study,” *Journal of Systems and Software*, vol. 203, p. 111749, Sep. 2023, doi: 10.1016/J.JSS.2023.111749.
- [10] A. Boronat and G. Fraser, “Fundamental Approaches to Software Engineering,” vol. 15693, p. Proceedings, 2025, doi: 10.1007/978-3-031-90900-9.
- [11] B. Pernici, P. Di Milano, I. Stefano, and D. Torre, “Deployment and Operation of Complex Software in Heterogeneous Execution Environments,” 2022, doi: 10.1007/978-3-031-04961-3.
- [12] T. Rosenberger, A. Knapp, and M. Roggenbach, “An Institutional Approach to Communicating UML State Machines,” *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*, vol. 13241 LNCS, pp. 205–224, 2022, doi: 10.1007/978-3-030-99429-7_12/SAVE-RESEARCH.
- [13] G. Munich and Manuel Wimmer, “Fundamental Approaches to Software Engineering,” vol. 15693, 2025, doi: 10.1007/978-3-031-90900-9.